Enroll

Browser

index.html • face-api.js 1.7.12

Models loaded and run here

Weights from the jsDelivr CDN, into the page.

Runs each frame: SSD MobileNet v1 (face box), 68-landmark net (eyes, nose, jaw).

Loaded, not run: faceRecognitionNet. No 128-D descriptor is computed.

Still in the browser, no model

Eye-aspect ratio and blink count.
Yaw from landmarks, binned into five sectors.
JPEG data URL of the frame (quality 0.8).

Live camera waits for all five sectors.
A still photo is one frontal JPEG only.

Shown on the page

Status and UIN.
The vector and the JPEG stay on the server.

Server

FastAPI • register_id/api.py

POST /api/enroll

uin
fullName
email

faceImage
JPEG data URL

faceViews, live only
sector, yaw, JPEG $\times 5$

embedding: not sent
landmarks: not sent

response

status, uin, fullName
cbeffGenerated

no embedding
no image

No neural model on the server

1. Build ISO 19794-5 / CBEFF XML from the JPEG bytes.
2. `embed_face_image`: SHA-256 of those bytes, mapped to a 128-D unit vector. Same JPEG, same vector.

A client-supplied vector is rejected.
One vector per sector. A photo stores sector `front`.

IDRepo REST write

identity row
uin, name, email
JPEG, CBEFF XML

template rows
uin, sector
128-D embedding

image stays in the identity row, not inside the vector

Database

PostgreSQL • via IDRepo REST

mosip_idrepo.citizen_identity

uin, full_name, email,
face_image (JPEG text),
cbeff_xml

mosip_ida.face_template

uin + sector,
embedding (JSON text)

FastAPI never opens SQL.
Mock IDRepo writes these tables.

1:1 authenticate

Same models, already in memory

Detection and landmarks run again
on the live frames or the uploaded photo.

faceRecognitionNet stays loaded and idle.
Blink totals and EAR min/max stay in the page.

Page keeps the liveness score

A still photo never calls checkLiveness.
It sets livenessPassed to true locally.

Then the page sends the probe

The same JPEG frames. No descriptor.

Shown on the page

Match, distance, score, and the citizen name.
No vector comes back.

POST /api/checkLiveness
live camera only

challengeId
blinkCount
minEar, maxEar
durationMs

no image
no embedding

A still photo skips this call
and sends livenessPassed true.

response

PASSED or FAILED
livenessScore
message

POST /api/authenticate

uin
livenessScore
livenessPassed

liveImage JPEG
liveViews, live only
sector, yaw, JPEG $\times 5$

embedding: not sent

response

verified, isFaceMatch
euclideanDistance
matchScore
citizen name and uin

no embedding
no image

evaluate_liveness

Thresholds on the numbers above:

at least one blink,
min EAR < 0.25,
EAR delta ≥ 0.045,
challenge nonce still live.

No model. No pixels.

Match, still no neural model

1. Read stored 128-D rows for this uin.
2. Hash each probe JPEG with the same function.
3. Euclidean distance in `matcher.py`.

Passes under 0.50, and only if liveness also passed.
The token in the response is a placeholder string.

IDRepo returns

identity
uin, name, email

face templates
sector + 128-D

nothing written on authenticate

Read only

`citizen_identity`
for the name and email.

`face_template`
for the stored vectors.

The probe JPEG is not saved.

neural net loaded or run server vector is a hash of the JPEG, not a face model solid arrow request dashed arrow response

Browser file: src/project/web/templates/index.html. Server file: src/register_id/api.py. Embedder: src/register_id/embedder.py. Tables: db/init-idrepo.sql. Click a box to open its function.

Functions behind the diagram

The line numbers are the line numbers in the source files. Click a box on the first page to land on its section. Where the running code has moved on from the label in the box, the note under the heading says what the function does now.

1 Browser models

Back to the diagram

SSD MobileNet v1 and the 68-landmark net are loaded in the page and run on each frame. faceRecognitionNet is loaded in the same call and is not used: `analyzeSource` asks only for a box and landmarks.

../../src/project/web/templates/index.html, lines 605–638

```
–••□□□–…□□□□□—…—Δ✓————————□
  async function loadModels() {
    if (modelsLoaded) return;
    if (modelsLoading) return modelsLoading;

    modelsLoading = (async () => {
      // Pin model weights to the same package version as the script above.
      const MODEL_URL = 'https://cdn.jsdelivr.net/npm/@vladmandic/face-api@1.7.12/model/';
      document.getElementById('cam-status').innerText = 'Loading face models...';
      try {
        if (typeof faceapi === 'undefined') {
          throw new Error('face-api.js failed to load from CDN');
        }
        await Promise.all([
          faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
          faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
          faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
        ]);
        modelsLoaded = true;
        document.getElementById('cam-status').innerText = 'Ready. Open Webcam or Upload Photo.';
        pushDiagEvent('Face-api models loaded');
        renderDiagnostics();
      } catch (err) {
        modelsLoaded = false;
        document.getElementById('cam-status').innerText = 'Model Load Error: ' + err.message;
        pushDiagEvent('Model load error: ' + err.message);
        renderDiagnostics();
        throw err;
      } finally {
        modelsLoading = null;
      }
    })();

    return modelsLoading;
  }
```

../../src/project/web/templates/index.html, lines 891–910

```
–••□□□–…□□□□□—…—Δ✓————————□–□□Δ…–□□
  async function analyzeSource(input, { live, generation }) {
    const width = overlay.width;
    const height = overlay.height;
    let stroke = '#ffc107';
    let drawList = [];
    const startedAt = performance.now();

    try {
      if (!modelsLoaded) {
        document.getElementById('cam-status').innerText = 'Face models not ready';
        return;
      }
      // Skip descriptors on the live path — they are unused and cost a lot of
      // frames, which makes short blinks easy to miss between samples.
      const rawList = await faceapi
        .detectAllFaces(input, new faceapi.SsdMobilenetv1Options({ minConfidence: 0.4 }))
        .withFaceLandmarks();
      if (generation !== frameGeneration) return;

      const resized = faceapi.resizeResults(rawList, { width, height });
```

2 Browser measurements and JPEG frames

Back to the diagram

Eye-aspect ratio, yaw, and the five sector bins are computed in the page from the landmarks. `commitSector` stores a JPEG data URL. `viewsPayload` is what later gets posted. No embedding is built here.

../../src/project/web/templates/index.html, lines 392–399

```
function captureVideoFrame() {
  if (!video.videoWidth) return null;
  const frame = document.createElement('canvas');
  frame.width = video.videoWidth;
  frame.height = video.videoHeight;
  frame.getContext('2d').drawImage(video, 0, 0);
  return frame.toDataURL('image/jpeg', 0.8);
}
```

../../src/project/web/templates/index.html, lines 427–433

```
function viewsPayload() {
  if (!liveSweepReady()) return null;
  return SWEEP_SECTORS.map((sector) => {
    const sample = sweepSamples[sector];
    return { sector: sample.sector, yaw: sample.yaw, image: sample.image };
  });
}
```

../../src/project/web/templates/index.html, lines 483–542

```
function estimateYaw(landmarks) {
  const jaw = landmarks.getJawOutline();
  const nose = landmarks.getNose();
  const tip = nose[3] || nose[Math.floor(nose.length / 2)];
  const imageLeft = jaw[0];
  const imageRight = jaw[jaw.length - 1];
  const leftDist = Math.hypot(tip.x - imageLeft.x, tip.y - imageLeft.y);
  const rightDist = Math.hypot(tip.x - imageRight.x, tip.y - imageRight.y);
  const span = leftDist + rightDist || 1;
  // Image-left is the subject's right on an unmirrored camera.
  // Negative yaw is the subject's own left, matching the Far left chip.
  const yaw = (rightDist - leftDist) / span;
  return Math.max(-1.2, Math.min(1.2, yaw));
}

function sectorForYaw(yaw) {
  if (yaw <= -0.42) return 'far-left';
  if (yaw <= -0.16) return 'left';
  if (yaw < 0.16) return 'front';
  if (yaw < 0.42) return 'right';
  return 'far-right';
}

function sectorLabel(sector) {
  return SECTOR_LABELS[sector] || sector;
}

function commitSector(sector, yaw) {
  if (pendingSector === sector) pendingStreak += 1;
  else {
    pendingSector = sector;
    pendingStreak = 1;
  }
  if (pendingStreak < 2) return;

  const image = captureVideoFrame();
  if (!image) return;
  const sample = { sector, yaw, image };
  const existing = sweepSamples[sector];
  let wroteFront = false;
  let wroteNew = false;
  if (!existing) {
    sweepSamples[sector] = sample;
    wroteFront = sector === 'front';
    wroteNew = true;
  } else if (sector === 'far-left' && yaw < existing.yaw) {
    sweepSamples[sector] = sample;
    wroteNew = true;
  } else if (sector === 'far-right' && yaw > existing.yaw) {
    sweepSamples[sector] = sample;
```

```
      wroteNew = true;
    } else if (sector === 'front' && Math.abs(yaw) < Math.abs(existing.yaw)) {
      sweepSamples[sector] = sample;
      wroteFront = true;
      wroteNew = true;
    }
    if (wroteFront) frontalSnapshot = sample.image;
    if (wroteNew) pushDiagEvent(`Sweep ${sector} @ yaw ${yaw.toFixed(3)}`);
    renderSweepChips();
  }
```

../../src/project/web/templates/index.html, lines 655–661

```
–••□□□–…□□□□□—…—Δ✓————————□
  function calculateEAR(eyeLandmarks) {
    const v1 = pointDist(eyeLandmarks[1], eyeLandmarks[5]);
    const v2 = pointDist(eyeLandmarks[2], eyeLandmarks[4]);
    const h = pointDist(eyeLandmarks[0], eyeLandmarks[3]);
    if (h === 0) return 0.0;
    return (v1 + v2) / (2.0 * h);
  }
```

3 Enroll request

Back to the diagram

`captureAndEnroll` posts the UIN, name, email, and JPEG data URLs. A live camera also sends `faceViews`. The body has no embedding field.

../../src/project/web/templates/index.html, lines 1168–1206

```
async function captureAndEnroll() {
  const faceViews = isLiveCamera() ? viewsPayload() : null;
  if (!faceCurrentlyAccepted || (!isLiveCamera() && !currentImageBase64)) {
    alert('No face detected! Please look at the camera or upload a clear photo.');
    return;
  }
  if (isLiveCamera() && !faceViews) {
    alert('Finish the head sweep before enrolling.');
    return;
  }
  const uin = document.getElementById('in-uin').value;
  const name = document.getElementById('in-name').value || 'Akraradet';
  const email = document.getElementById('in-email').value || 'citizen@ait.asia';

  let faceImage = currentImageBase64;
  if (isLiveCamera()) {
    faceImage = frontalSnapshot || (sweepSamples.front && sweepSamples.front.image) || captureVideoFrame();
  }

  const resDiv = document.getElementById('enroll-result');
  resDiv.classList.remove('d-none');
  resDiv.className = 'alert alert-info small mt-2';
  resDiv.innerHTML = '<span>Saving citizen record and serializing ISO 19794-5 CBEFF XML...</span>';

  const enrollPayload = {
    uin: uin,
    fullName: name,
    email: email,
    faceImage: faceImage || ''
  };
  if (faceViews) enrollPayload.faceViews = faceViews;

  try {
    const res = await fetch('/api/enroll', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(enrollPayload)
    });
    const data = await res.json();
```

4 Server enroll

Back to the diagram

`enroll_citizen_endpoint` builds the CBEFF XML, then hashes each JPEG. `embed_face_image` is that hash. It is not a face model. `_templates_for_enrollment` refuses to trust a vector that arrived from the browser.

../../src/register_id/api.py, lines 38–73

```
def _embed_or_422(image: str) -> list[float]:
    try:
        return embed_face_image(image)
    except ValueError as exc:
        raise HTTPException(status_code=422, detail=str(exc)) from exc


def _pose_distance(left: list[float], right: list[float]) -> float:
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(left, right)))


def _templates_for_enrollment(req: EnrollRequest) -> list[dict]:
    """
    Build server-side templates from image bytes.

    A head sweep stores one vector per sector. The frontal template is the
    front sample. A single portrait stores sector "front" from faceImage.
    """
    if req.faceViews is not None:
        by_sector: dict[str, list[float]] = {}
        for sample in req.faceViews:
            by_sector[sample.sector] = _embed_or_422(sample.image)

        front = by_sector["front"]
        for sector in ("far-left", "far-right"):
            if _pose_distance(front, by_sector[sector]) < 0.05:
                raise HTTPException(
                    status_code=422,
                    detail=f"{sector} sample is not a distinct pose from the frontal sample",
                )
        return [
            {"sector": sample.sector, "embedding": by_sector[sample.sector]}
            for sample in req.faceViews
        ]

    return [{"sector": "front", "embedding": _embed_or_422(req.faceImage)}]
```

../../src/register_id/api.py, lines 146–191

```

@router.post("/enroll")
def enroll_citizen_endpoint(req: EnrollRequest):
    """
    Enrolls a citizen face through the MOSIP IDRepo REST interface:
    1. Extracts JPEG image and serializes into ISO 19794-5 / ISO 19785-3 CBEFF XML.
    2. Computes 128-D templates from the image bytes. Client vectors are rejected.
    3. Persists demographics on the identity and templates on the IDA table.

    Keycloak login is a separate POST /api/credentials call. No password is accepted here.
    """
    idrepo = get_idrepo_client()

    cbeff_xml = None
    try:
        jpeg_bytes = extract_jpeg_bytes(req.faceImage)
        cbeff_xml = build_cbeff_xml(uin=req.uin, jpeg_bytes=jpeg_bytes)
    except Exception as e:
        logger.warning(f"Could not construct CBEFF XML (storing raw data): {e}")

    templates = _templates_for_enrollment(req)
    saved = idrepo.add_identity(
        uin=req.uin,
        full_name=req.fullName,
        email=req.email,
        face_image=req.faceImage,
        cbeff_xml=cbeff_xml,
        templates=templates,
    )
    if not saved:
        error_msg = getattr(idrepo, "last_error", None) or "Failed to persist identity in MOSIP IDRepo"
        logger.error(f"Enrollment failed in IDRepo: {error_msg}")
        raise HTTPException(
            status_code=502,
            detail=f"MOSIP IDRepo enrollment failed: {error_msg}",
```

```
        )

    return {
        "status": "SUCCESS",
        "message": "Citizen face enrolled successfully via MOSIP IDRepo interface",
        "uin": req.uin,
        "fullName": req.fullName,
        "backend": "mosip-idrepo-rest",
        "cbeffGenerated": cbeff_xml is not None,
    }
```

../../src/register_id/embedder.py, lines 15–44

```
def embed_face_image(data_url_or_base64: str) -> list[float]:
    """
    Hash image bytes into a 128-D unit vector in [-1, 1].

    Empty input, undecodable input, and empty decoded bytes raise ValueError.
    """
    if not isinstance(data_url_or_base64, str) or not data_url_or_base64.strip():
        raise ValueError("face image is empty")

    try:
        data = extract_jpeg_bytes(data_url_or_base64)
    except Exception as exc:
        raise ValueError("face image could not be decoded") from exc

    if not data:
        raise ValueError("face image is empty")

    raw = bytearray()
    counter = 0
    while len(raw) < 128:
        raw.extend(hashlib.sha256(data + counter.to_bytes(4, "big")).digest())
        counter += 1

    # Map each hash byte onto [-1, 1]. 127.5 is not an integer, so no byte
    # lands on exactly zero and the pre-normalization vector cannot vanish.
    coords = [(byte / 127.5) - 1.0 for byte in raw[:128]]
    norm = math.sqrt(sum(value * value for value in coords))
    if norm == 0.0:
        raise ValueError("face embedding norm is zero")
    return [value / norm for value in coords]
```

5 Database write

Back to the diagram

The gateway posts to IDRepo. The mock service writes the JPEG and CBEFF onto `citizen_identity`, and the 128-D vectors onto `face_template`. FastAPI does not open SQL.

../../src/register_id/idrepo_client.py, lines 154–241

```
    def add_identity(
        self,
        uin: str,
        full_name: str,
        email: str | None = None,
        face_image: str | None = None,
        cbeff_xml: str | None = None,
        templates: list | None = None,
    ) -> bool:
        """
        Enrolls or updates an identity in MOSIP IDRepo via POST /idrepository/v1/identity/.

        Demographics stay in request.identity. Match vectors travel in request.templates
        and are not written onto the identity row.
        """
        documents = []
        if cbeff_xml:
            cbeff_b64url = base64.urlsafe_b64encode(cbeff_xml.encode("utf-8")).decode("utf-8")
            documents.append({
                "category": "individualBiometrics",
                "value": cbeff_b64url,
            })

        request_body: dict[str, Any] = {
            "registrationId": self._make_rid(),
            "status": "ACTIVATED",
            "identity": {
                "UIN": uin,
                "fullName": [{"language": "eng", "value": full_name}],
                "email": email,
                "faceImage": face_image,
            },
            "documents": documents,
        }
        if templates is not None:
            request_body["templates"] = templates

        payload = {
            "id": "mosip.id.create",
            "version": "v1",
            "requesttime": self._now_iso(),
            "metadata": {},
            "request": request_body,
        }

        self.last_error = None
        remote_error = None

        # 1. Attempt remote HTTP call
        try:
            r = requests.post(
                f"{self.base_url}/idrepository/v1/identity/",
                json=payload,
                headers={"Content-Type": "application/json"},
                timeout=5,
            )
            if r.status_code in (200, 201):
                data = r.json()
                if not data.get("errors"):
                    return True
                errors = data.get("errors", [])
                err_msgs = [e.get("message", str(e)) for e in errors]
                remote_error = f"MOSIP IDRepo error: {', '.join(err_msgs)}"
            else:
                remote_error = f"MOSIP IDRepo returned HTTP {r.status_code}: {r.text[:200]}"
            logger.warning(f"IDRepo remote enroll failed: {remote_error}")
        except Exception as e:
            remote_error = f"Failed to connect to MOSIP IDRepo at '{self.base_url}': {e}"
            logger.debug(f"IDRepo remote connection failed: {e}")

        # 2. In-process fallback
        if self.enable_inprocess_fallback:
            try:
                client = self._get_test_client()
```

```
                r = client.post("/idrepository/v1/identity/", json=payload)
                if r.status_code in (200, 201):
                    data = r.json()
                    if not data.get("errors"):
                        return True
                    errors = data.get("errors", [])
                    err_msgs = [e.get("message", str(e)) for e in errors]
                    self.last_error = f"In-process mock IDRepo error: {', '.join(err_msgs)}"
                    return False
            except Exception as e:
                logger.debug(f"In-process IDRepo fallback failed: {e}")

        self.last_error = remote_error or "Unknown error communicating with MOSIP IDRepo"
        return False
```

../../src/mock/idrepo/storage.py, lines 173–267

```
    def save_identity(
        self,
        uin: str,
        full_name: str,
        email: str | None,
        face_image: str | None,
        cbeff_xml: str | None,
    ) -> bool:
        if self.active_backend == "postgres":
            conn = self._get_pg_connection()
            if conn:
                try:
                    with conn.cursor() as cur:
                        cur.execute("""
                            INSERT INTO mosip_idrepo.citizen_identity
                                (uin, full_name, email, face_image, cbeff_xml)
                            VALUES (%s, %s, %s, %s, %s)
                            ON CONFLICT (uin) DO UPDATE
                            SET full_name = EXCLUDED.full_name,
                                email = EXCLUDED.email,
                                face_image = EXCLUDED.face_image,
                                cbeff_xml = EXCLUDED.cbeff_xml;
                        """, (uin, full_name, email, face_image, cbeff_xml))
                        self._clear_legacy_pg(cur, uin)
                    conn.commit()
                    return True
                except Exception as e:
                    logger.error(f"PostgreSQL insert failed: {e}")
                    conn.rollback()
                finally:
                    conn.close()

        conn = self._get_sqlite_connection()
        try:
            with conn:
                conn.execute("""
                    INSERT INTO citizen_identity
                        (uin, full_name, email, face_image, cbeff_xml)
                    VALUES (?, ?, ?, ?, ?)
                    ON CONFLICT (uin) DO UPDATE
                    SET full_name = excluded.full_name,
                        email = excluded.email,
                        face_image = excluded.face_image,
                        cbeff_xml = excluded.cbeff_xml;
                """, (uin, full_name, email, face_image, cbeff_xml))
                self._clear_legacy_sqlite(conn, uin)
            return True
        except Exception as e:
            logger.error(f"SQLite insert failed: {e}")
            return False
        finally:
            conn.close()

    def save_face_templates(self, uin: str, templates: list[dict]) -> bool:
        """Replace every stored sector for this UIN in one transaction."""
        rows = [
            (uin, item["sector"], _embedding_text(item["embedding"]))
            for item in templates
        ]

        if self.active_backend == "postgres":
            conn = self._get_pg_connection()
            if conn:
                try:
```

```
                    with conn.cursor() as cur:
                        cur.execute(
                            "DELETE FROM mosip_ida.face_template WHERE uin = %s",
                            (uin,),
                        )
                        cur.executemany("""
                            INSERT INTO mosip_ida.face_template (uin, sector, embedding)
                            VALUES (%s, %s, %s)
                        """, rows)
                    conn.commit()
                    return True
                except Exception as e:
                    logger.error(f"PostgreSQL template replace failed: {e}")
                    conn.rollback()
                finally:
                    conn.close()

        conn = self._get_sqlite_connection()
        try:
            with conn:
                conn.execute("DELETE FROM face_template WHERE uin = ?", (uin,))
                conn.executemany("""
                    INSERT INTO face_template (uin, sector, embedding)
                    VALUES (?, ?, ?)
                """, rows)
            return True
        except Exception as e:
            logger.error(f"SQLite template replace failed: {e}")
            return False
        finally:
            conn.close()
```

../../db/init-idrepo.sql, lines 1–29

```
-- ============================================================================
-- MOSIP IDRepo Database Schema Initialization
-- Executed automatically on first container startup of PostgreSQL
-- ============================================================================

CREATE SCHEMA IF NOT EXISTS mosip_idrepo;

CREATE TABLE IF NOT EXISTS mosip_idrepo.citizen_identity (
    uin VARCHAR(32) PRIMARY KEY,
    full_name VARCHAR(128) NOT NULL,
    email VARCHAR(128),
    face_image TEXT,
    cbeff_xml TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_citizen_created_at
    ON mosip_idrepo.citizen_identity (created_at DESC);

-- Match vectors live outside the identity row. IDA reads this table only.
CREATE SCHEMA IF NOT EXISTS mosip_ida;

CREATE TABLE IF NOT EXISTS mosip_ida.face_template (
    uin VARCHAR(32) NOT NULL,
    sector VARCHAR(16) NOT NULL,
    embedding TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (uin, sector)
);
```

6 Enroll response

Back to the diagram

The endpoint returns status, UIN, and name. The vector and the JPEG are not in that JSON.

../../src/register_id/api.py, lines 183–190

```
    return {
        "status": "SUCCESS",
        "message": "Citizen face enrolled successfully via MOSIP IDRepo interface",
        "uin": req.uin,
        "fullName": req.fullName,
        "backend": "mosip-idrepo-rest",
        "cbeffGenerated": cbeff_xml is not None,
    }
```

7 Liveness numbers

Back to the diagram

`evaluate_liveness` is still the body of `POST /api/checkLiveness`: blink count and eye-aspect thresholds, no pixels. On the authenticate path the page does not call that route. Opening the camera posts `/api/sweep/challenge` instead, and the later authenticate body sends `livenessPassed: false`.

../../src/register_id/api.py, lines 139–144

```
def check_liveness_endpoint(req: LivenessRequest):
    """
    MOSIP BioSDK SPI Contract: IBioApi.checkLiveness() (ISO/IEC 30107-3 PAD)
    Evaluates Eye Aspect Ratio (EAR) dynamics and blink events.
    """
    return evaluate_liveness(req)
```

../../src/register_id/liveness.py, lines 54–106

```
def evaluate_liveness(req: LivenessRequest) -> LivenessResponse:
    """
    Evaluates client EAR dynamics according to ISO 30107-3 PAD rules.
    A live human presents natural micro-motion and eye closure:
    - At least 1 natural blink cycle
    - Minimum EAR observed < 0.25 (eye closure; face-api troughs are shallow)
    - EAR delta (max - min) >= 0.045 (transition dynamic)
    - If challengeId is supplied, validates active anti-replay challenge satisfaction.
    """
    ear_delta = req.maxEar - req.minEar
    is_live = req.blinkCount >= 1 and req.minEar < 0.25 and ear_delta >= 0.045

    if not is_live:
        return LivenessResponse(
            status="FAILED",
            livenessScore=12.0,
            message="Presentation Attack / Spoof Detected (Static photo or insufficient motion).",
        )

    # Validate active challenge if provided
    if req.challengeId:
        ch = _ACTIVE_CHALLENGES.get(req.challengeId)
        if not ch:
            return LivenessResponse(
                status="FAILED",
                livenessScore=20.0,
                message="Challenge expired or invalid. Please request a new liveness challenge.",
            )

        now = time.time()
        if now > ch["expiresAt"]:
            return LivenessResponse(
                status="FAILED",
                livenessScore=25.0,
                message="Challenge timed out. Please complete the action within the allotted time limit.",
            )

        if req.blinkCount < ch["targetBlinks"]:
            return LivenessResponse(
                status="FAILED",
                livenessScore=30.0,
                message=f"Challenge incomplete: observed {req.blinkCount} blinks, expected {ch['targetBlinks']}.",
            )

        # Nonce burning: single-use token consumed upon successful verification
        _ACTIVE_CHALLENGES.pop(req.challengeId, None)

    confidence = min(99.0, max(85.0, (1.0 - req.minEar) * 100.0))
    return LivenessResponse(
        status="PASSED",
        livenessScore=round(confidence, 1),
        message="Human liveness verified. No static presentation attack detected.",
    )
```

../../src/project/web/templates/index.html, lines 581–601

```
−••□□□−…□□□□□—…—Δ✓——————————
  async function fetchLivenessChallenge() {
    try {
      const res = await fetch('/api/sweep/challenge', { method: 'POST' });
      if (res.ok) {
        activeChallenge = await res.json();
        challengeBlinks = 0;
        diag.challengeBlinks = 0;
```

```
        diag.challengeTarget = 0;
        const badge = document.getElementById('challenge-badge');
        badge.classList.remove('d-none');
        badge.className = 'position-absolute bottom-0 start-0 m-3 px-3 py-2 rounded bg-black bg-opacity-80 text-warning font-monospace small border border-warning';
        document.getElementById('challenge-icon').innerText = '';
        document.getElementById('challenge-text').innerText = activeChallenge.prompt;
        pushDiagEvent(`Sweep challenge ${activeChallenge.timeLimitSeconds}s`);
        renderDiagnostics();
      }
    } catch (e) {
      console.warn('Could not fetch sweep challenge:', e);
      pushDiagEvent('Sweep challenge fetch failed');
    }
  }
```

../../src/register_id/api.py, lines 94–99

```
@router.post("/sweep/challenge", response_model=SweepChallenge)
def create_sweep_challenge_endpoint():
    """
    Issues a single-use head-sweep challenge. Blink score is not part of verification.
    """
    return create_sweep_challenge()
```

../../src/register_id/sweep.py, lines 42–63

```
def create_sweep_challenge() -> SweepChallenge:
    now = time.time()
    expired = [key for key, item in _CHALLENGES.items() if item["expiresAt"] < now - 60]
    for key in expired:
        _CHALLENGES.pop(key, None)

    challenge_id = f"sweep_{secrets.token_hex(8)}"
    _CHALLENGES[challenge_id] = {
        "createdAt": now,
        "expiresAt": now + SWEEP_TTL_SECONDS,
        "used": False,
    }
    return SweepChallenge(
        challengeId=challenge_id,
        timeLimitSeconds=SWEEP_TTL_SECONDS,
        prompt=(
            "Turn your head from your left shoulder to your right. "
            f"Finish within {SWEEP_TTL_SECONDS} seconds."
        ),
        createdAt=now,
    )
```

8 Authenticate request

Back to the diagram

The page posts the UIN, the sweep challenge id, and the five JPEG views. `livenessScore` is 0 and `livenessPassed` is false. No embedding is sent.

../../src/project/web/templates/index.html, lines 1290–1328

```
–••□□□–…□□□□□—…—Δ✓————————□–□□Δ…–□□———□□□□□
  async function captureAndAuthenticate() {
    const liveViews = isLiveCamera() ? viewsPayload() : null;
    if (!isLiveCamera()) {
      alert('Verification needs a live five-pose head sweep.');
      return;
    }
    if (!faceCurrentlyAccepted || !liveViews) {
      alert('Finish the head sweep before verifying.');
      return;
    }
    if (!activeChallenge || !activeChallenge.challengeId) {
      alert('The sweep challenge is missing. Open the camera again.');
      return;
    }
    const uin = document.getElementById('sel-uin').value;
    const resDiv = document.getElementById('auth-result');
    resDiv.classList.remove('d-none');
    resDiv.innerHTML = '<div class="alert alert-info small">Checking the five-pose …sweep</div>';

    diag.match.status = 'checking';
    diag.match.message = null;
    renderDiagnostics();

    const authPayload = {
      uin: uin,
      livenessScore: 0,
      livenessPassed: false,
      challengeId: activeChallenge.challengeId,
      liveViews: liveViews
    };
    const liveImage = frontalSnapshot || (sweepSamples.front && sweepSamples.front.image);
    if (liveImage) authPayload.liveImage = liveImage;

    try {
      const authRes = await fetch('/api/authenticate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(authPayload)
      });
```

9 Server check of the five images

Back to the diagram

`POST /api/authenticate` calls `assess_sweep`. That check hashes each JPEG only to reject a repeated picture, and it checks that the client yaw labels run left to right. It does not compare the probe with the stored gallery, and it does not treat the result as a face match. `isFaceMatch` is returned false.

../../src/register_id/api.py, lines 219–254

```
@router.post("/authenticate", response_model=AuthResponse)
def authenticate_citizen_endpoint(req: AuthRequest):
    """
    Verify a five-pose head sweep bound to an expiring challenge.

    Blink flags on the request are ignored. The hash embedder is not a face
    model, so a passing sweep is a submission check, not an identity match
    and not proof of a live head turn.
    """
    idrepo = get_idrepo_client()
    citizen = idrepo.get_identity(req.uin)
    if not citizen:
        raise HTTPException(status_code=404, detail=f"Citizen UIN '{req.uin}' not found in registry.")

    assessment = assess_sweep(req.challengeId, req.liveViews)
    verified = assessment.accepted

    return AuthResponse(
        status="SUCCESS" if verified else "FAILED",
        verified=verified,
        isFaceMatch=False,
        livenessPassed=False,
        euclideanDistance=1.0,
        matchScore=0.0,
        livenessScore=0.0,
        threshold=MATCH_THRESHOLD,
        citizen={
            "uin": citizen["uin"],
            "fullName": citizen["full_name"],
            "email": citizen.get("email"),
        },
        jwtToken=None,
        message=assessment.message,
        sweepAccepted=assessment.accepted,
        yawSource=assessment.yaw_source,
    )
```

../../src/register_id/sweep.py, lines 31–39

```
def estimate_yaw_from_image(image: str) -> float | None:
    """
    Yaw from the image itself.

    No landmark model is wired in, so this always returns None. Callers must
    not treat client-supplied yaw as a measurement from the pixels.
    """
    del image
    return None
```

../../src/register_id/sweep.py, lines 90–162

```
def _distinct_images(views: list[FaceViewSample]) -> str | None:
    by_sector = {sample.sector: embed_face_image(sample.image) for sample in views}
    sectors = list(FACE_VIEW_SECTORS)
    for index, left in enumerate(sectors):
        for right in sectors[index + 1 :]:
            distance = compute_euclidean_distance(by_sector[left], by_sector[right])
            if distance < MIN_IMAGE_DISTANCE:
                return (
                    "Sweep rejected: the five images are not distinct. "
                    "The same picture was submitted for more than one pose."
                )
    return None


def assess_sweep(
    challenge_id: str | None,
    views: list[FaceViewSample] | None,
    now: float | None = None,
) -> SweepAssessment:
    """
    Accept a sweep submission or explain why it was rejected.

```

```
    Server-side yaw is attempted and, while it is unavailable, client yaw is
    used only as a label-order check. Acceptance is not a liveness result.
    """
    if not views:
        return SweepAssessment(
            accepted=False,
            message=(
                "Head sweep required: submit liveViews covering far-left, left, "
                "front, right, and far-right."
            ),
            yaw_source="none",
        )

    clock = time.time() if now is None else now
    challenge_error = _consume(challenge_id, clock)
    if challenge_error:
        return SweepAssessment(accepted=False, message=challenge_error, yaw_source="none")

    server_yaws = [estimate_yaw_from_image(sample.image) for sample in views]
    yaw_source = "server" if all(yaw is not None for yaw in server_yaws) else "client"
    if yaw_source == "server":
        measured = [
            FaceViewSample(sector=sample.sector, yaw=yaw, image=sample.image)
            for sample, yaw in zip(views, server_yaws)
        ]
        order_error = _yaw_order(measured)
    else:
        order_error = _yaw_order(views)
    if order_error:
        return SweepAssessment(accepted=False, message=order_error, yaw_source=yaw_source)

    try:
        distinct_error = _distinct_images(views)
    except ValueError as exc:
        return SweepAssessment(accepted=False, message=str(exc), yaw_source=yaw_source)
    if distinct_error:
        return SweepAssessment(accepted=False, message=distinct_error, yaw_source=yaw_source)

    if yaw_source == "client":
        message = (
            "Sweep submission accepted: five distinct images with client yaw in "
            "left-to-right order. Yaw was not estimated from the image pixels, "
            "so this does not show a live head turn. Identity was not compared; "
            "no face model is configured."
        )
    else:
        message = (
            "Sweep submission accepted from server-estimated yaw. "
            "Identity was not compared; no face model is configured."
        )
    return SweepAssessment(accepted=True, message=message, yaw_source=yaw_source)
```

../../src/register_id/matcher.py, lines 11–54

```
def compute_euclidean_distance(embedding1: list[float], embedding2: list[float]) -> float:
    """
    Computes Euclidean (L2) Distance between two embedding vectors.
    """
    if len(embedding1) != len(embedding2):
        raise ValueError(
            f"Embedding dimension mismatch: {len(embedding1)} vs {len(embedding2)}"
        )
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(embedding1, embedding2)))


def compute_cosine_similarity(embedding1: list[float], embedding2: list[float]) -> float:
    """
    Computes Cosine Similarity between two embedding vectors (-1.0 to 1.0).
    """
    if len(embedding1) != len(embedding2):
        raise ValueError(
            f"Embedding dimension mismatch: {len(embedding1)} vs {len(embedding2)}"
        )
    dot_product = sum(a * b for a, b in zip(embedding1, embedding2))
    norm_a = math.sqrt(sum(a * a for a in embedding1))
    norm_b = math.sqrt(sum(b * b for b in embedding2))
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    return dot_product / (norm_a * norm_b)


```

```
def match_embeddings(
    stored: list[float],
    probe: list[float],
    threshold: float = 0.50,
) -> tuple[bool, float, float]:
    """
    Compares stored gallery embedding against live probe.

    Returns:
        (is_match: bool, distance: float, match_confidence_pct: float)
    """
    distance = compute_euclidean_distance(stored, probe)
    is_match = distance < threshold
    # Normalize score: distance 0.0 -> 100%, distance >= 1.0 -> 0%
    confidence = max(0.0, min(100.0, (1.0 - (distance / 1.0)) * 100.0))
    return is_match, round(distance, 4), round(confidence, 1)
```

10 Database read

Back to the diagram

Authenticate loads the identity row so the response can name the citizen. `get_face_templates` is the read of the stored vectors. The authenticate endpoint above does not call it.

../../src/register_id/idrepo_client.py, lines 243–265

```
    def get_identity(self, uin: str) -> dict[str, Any] | None:
        """
        Retrieves a citizen identity from MOSIP IDRepo via GET /idrepository/v1/identity/uin/{uin}.
        """
        # 1. Attempt remote HTTP call
        try:
            r = requests.get(f"{self.base_url}/idrepository/v1/identity/uin/{uin}", timeout=3)
            if r.status_code == 200:
                return _record_from_read_response(r.json(), uin)
        except Exception as e:
            logger.debug(f"IDRepo remote get failed: {e}")

        # 2. In-process fallback
        if self.enable_inprocess_fallback:
            try:
                client = self._get_test_client()
                r = client.get(f"/idrepository/v1/identity/uin/{uin}")
                if r.status_code == 200:
                    return _record_from_read_response(r.json(), uin)
            except Exception as e:
                logger.error(f"In-process IDRepo fallback get failed: {e}")

        return None
```

../../src/register_id/idrepo_client.py, lines 267–290

```
    def get_face_templates(self, uin: str) -> list[dict[str, Any]]:
        """
        Load match vectors from GET /ida/v1/face-template/uin/{uin}.

        Remote is tried first, then the in-process mock. An empty list means
        the UIN has no templates (or both calls failed).
        """
        try:
            r = requests.get(f"{self.base_url}/ida/v1/face-template/uin/{uin}", timeout=3)
            if r.status_code == 200:
                return _templates_from_body(r.json())
        except Exception as e:
            logger.debug(f"IDRepo remote face-template get failed: {e}")

        if self.enable_inprocess_fallback:
            try:
                client = self._get_test_client()
                r = client.get(f"/ida/v1/face-template/uin/{uin}")
                if r.status_code == 200:
                    return _templates_from_body(r.json())
            except Exception as e:
                logger.error(f"In-process IDRepo fallback face-template get failed: {e}")

        return []
```

../../src/mock/idrepo/storage.py, lines 279–308

```
    def get_face_templates(self, uin: str) -> list[dict[str, Any]]:
        if self.active_backend == "postgres":
            conn = self._get_pg_connection()
            if conn:
                try:
                    from psycopg2.extras import RealDictCursor
                    with conn.cursor(cursor_factory=RealDictCursor) as cur:
                        cur.execute("""
                            SELECT sector, embedding
                            FROM mosip_ida.face_template
                            WHERE uin = %s
                            ORDER BY sector;
                        """, (uin,))
                        return self._rows_to_templates(cur.fetchall())
                except Exception as e:
                    logger.error(f"PostgreSQL template select failed: {e}")
                finally:
                    conn.close()

        conn = self._get_sqlite_connection()
        try:
```

```
            cur = conn.cursor()
            cur.execute("""
                SELECT sector, embedding
                FROM face_template
                WHERE uin = ?
                ORDER BY sector;
            """, (uin,))
            return self._rows_to_templates(cur.fetchall())
        finally:
```

11 Authenticate response

Back to the diagram

The page shows the sweep message. The JSON still carries distance and score fields; the endpoint fills distance with 1 and the score with 0, and it does not return an embedding or an image.

../../src/register_id/api.py, lines 236–254

```
    return AuthResponse(
        status="SUCCESS" if verified else "FAILED",
        verified=verified,
        isFaceMatch=False,
        livenessPassed=False,
        euclideanDistance=1.0,
        matchScore=0.0,
        livenessScore=0.0,
        threshold=MATCH_THRESHOLD,
        citizen={
            "uin": citizen["uin"],
            "fullName": citizen["full_name"],
            "email": citizen.get("email"),
        },
        jwtToken=None,
        message=assessment.message,
        sweepAccepted=assessment.accepted,
        yawSource=assessment.yaw_source,
    )
```

../../src/project/web/templates/index.html, lines 1329–1370

```
–••□□□–…□□□□□—…—Δ✓————————□–□□Δ…–□□———□□□□□…
      const data = await authRes.json();
      diag.match.status = data.sweepAccepted ? 'sweep accepted' : 'rejected';
      diag.match.isFaceMatch = data.isFaceMatch;
      diag.match.verified = data.verified;
      diag.match.distance = data.euclideanDistance;
      diag.match.matchScore = data.matchScore;
      diag.match.threshold = data.threshold != null ? data.threshold : 0.50;
      diag.match.message = data.message || data.detail || null;
      diag.match.padPassed = false;
      diag.match.padScore = 0;
      diag.match.padMessage = 'Blink is not used for this decision';
      pushDiagEvent(
        `Sweep ${data.sweepAccepted ? 'accepted' : 'rejected'} . face match ${data.isFaceMatch ? 'YES' : 'NO'} . yaw ${data.yawSource || –''}`
      );
      renderDiagnostics();
      const serverNote = data.message
        ? escapeHtml(data.message)
        : escapeHtml(data.detail || 'Sweep was not accepted.');

      if (data.sweepAccepted) {
        resDiv.innerHTML = `
          <div class="alert alert-success border-success p-3">
            <div class="fw-bold mb-2">Sweep submission accepted</div>
            <div class="small">${serverNote}</div>
          </div>
        `;
      } else {
        resDiv.innerHTML = `
          <div class="alert alert-danger border-danger p-3">
            <div class="fw-bold mb-2">Sweep not accepted</div>
            <div class="small">${serverNote}</div>
          </div>
        `;
      }
    } catch (e) {
      diag.match.status = 'error';
      diag.match.message = e.message;
      pushDiagEvent('Auth error: ' + e.message);
      renderDiagnostics();
      resDiv.innerHTML = `<div class="alert alert-danger small">Authentication error: ${escapeHtml(e.message)}</div>`;
    }
  }
```